

Scenario-1: Search Functionality

Normal Prompt: "Test the search feature."

What's missing

No product type (e-commerce, hospital), no filter/sort behavior, no empty/invalid input scenarios, no performance angle.

Prompt using ICEPOT:

Instruction: As a Senior Test Engineer to write test script for an ecommerce application

Context: Test the Search feature of an e-commerce application

[MANDATORY] Generate test cases for ecommerce application

[MANDATORY] Generate only 15 test cases

[MANDATORY] Include functional and non functional like performance and security

[MANDATORY] Include positive and negative test cases

[MANDATORY] Include filter

Example:

TC ID | Description | PreConditions | Steps | Actual Result | Expected Result

Persona:

Act as a test lead to review the test cases

Output:

Downloadable excel format

Tone:

Professional Tone with clarity and concise

OBSERVATIONS:

1. **Structured Test Design** – The output followed a clear tabular format with TC ID, Description, Preconditions, Steps, Actual Result, and Expected Result. This structure made the test cases easy to read, review, and directly usable in documentation. The improvement came from explicit formatting instructions and mandatory fields.
2. **Comprehensive Coverage** – The test cases included functional, non-functional (performance and security), positive, negative, and filter scenarios. This ensured the search feature was tested from multiple angles, not just basic functionality. The improvement was driven by mandatory inclusion of diverse case types.
3. **Professional Tone & Usability** – The output was concise, professional, and aligned with the persona of a Senior Test Engineer/Test Lead. It was also tailored for practical use (Excel format), making it suitable for real QA workflows. This improvement resulted from persona, tone, and output requirements embedded in the prompt.

Together, these enhancements made the ICEPOT-style prompt produce outputs that were **clear, complete, and actionable** rather than vague or minimal.

SCENARIO-2: API Testing

Normal Prompt: "Write API test cases for user registration."

What's missing

No HTTP method, no request/response structure, no status codes to validate, no tool context (Postman/RestAssured).

Prompt using ICEPOT:

Instruction: As a Senior Test Engineer to Write API test cases for user registration

Context: Test the API features for user registration using Postman

[MANDATORY] Generate test cases for API features

[MANDATORY] Generate only 15 test cases

[MANDATORY] Include Status Code Validation

[MANDATORY] Include positive and negative test cases

[MANDATORY] Include request/response structure

Example:

TC ID | Description | PreConditions | Steps | Actual Result | Expected Result

Persona:

Act as a test lead to review the test cases

Output:

Downloadable excel format

Tone:

Professional Tone with clarity and concise

OBSERVATIONS:

- 1) **Contextual Relevance** – By specifying Postman as the testing tool, the output aligned with real-world QA workflows, including practical steps like sending requests, checking headers, and validating responses. This improvement came from contextual framing in the prompt.
- 2) **Explicit Validation Focus** – The output included mandatory **status code validation** (e.g., 200, 400, 401, 500), ensuring each test case checked not just functional behavior but also proper API response handling. This improvement came from explicitly requiring validation in the prompt.
- 3) **Request/Response Structure Testing** – Each test case documented the expected request payload and response body, making it easier to verify schema compliance and

detect malformed data. This improvement was driven by the mandatory inclusion of request/response structure.

Together, these enhancements made the test cases **more rigorous, realistic, and aligned with industry best practices**.

SCENARIO-3: Mobile App — OTP Login

Normal Prompt: Test OTP login on mobile.

What's missing

No platform (Android/iOS), no OTP expiry time, no resend behavior, no network failure scenario, no Appium/device context.

Prompt using ICEPOT:

Instruction: As a Senior Test Engineer to Test OTP login on mobile

Context: Test the Test OTP login on mobile

[MANDATORY] Generate test cases for OTP login on mobile

[MANDATORY] Generate only 15 test cases

[MANDATORY] Include Platform

[MANDATORY] Include OTP expiry time

[MANDATORY] Include positive and negative test cases

[MANDATORY] Include Appium device context

Example:

TC ID | Description | PreConditions | Steps | Actual Result | Expected Result

Persona:

Act as a test lead to review the test cases

Output:

Downloadable excel format

Tone:

Professional Tone with clarity and concise

OBSERVATIONS:

1. **Platform-Specific Detailing** – The output explicitly included mobile platforms (Android, iOS), ensuring test cases were relevant to device contexts rather than generic. This improvement came from the mandatory requirement to include platform details.

2. **Device Context Integration** – By embedding Appium device context, the test cases aligned with real automation workflows, including steps like launching the app, interacting with UI elements, and validating responses. This improvement came from contextual framing in the prompt.
3. **Time-Sensitive Validation** – Test cases covered OTP expiry scenarios (e.g., OTP valid for 60 seconds, expired OTP rejection), which are critical for security and usability. This improvement was driven by the explicit requirement to include OTP expiry time.

Together, these enhancements made the test cases **more rigorous, realistic, and aligned with mobile QA best practices**, ensuring both functional correctness and secure handling of OTP login